

Enabling standardized, efficient system level test campaigns via the Ingenium electronic procedure system.

Christopher Swan
Jet Propulsion Laboratory,
California Institute of Technology
4800 Oak Grove Dr.
Pasadena, CA 91109
Christopher.A.Swan@jpl.nasa.gov

Hongman Kim
Jet Propulsion Laboratory,
California Institute of Technology
4800 Oak Grove Dr.
Pasadena, CA 91109
Hongman.Kim@jpl.nasa.gov

Abstract— The Ingenium electronic procedure system was developed at the Jet Propulsion Laboratory to both standardize and streamline system level test campaigns. This software suite allows integration and test engineers to develop test procedures that can be executed on a variety of test venues, from simulators to flight vehicles. Ingenium executes these procedures electronically by directly interfacing with the ground system and can support both manual and fully automatic modes. These procedures are developed within the Ingenium web application and provide easy integration with both spacecraft dictionaries as well as Verification and Validation (V&V) artifacts. The standard format, both visually and electronically, supports flattening the learning curve for personnel as well as enabling standard tooling for augmenting the system for project specific needs. Ingenium has been deployed on the Psyche and Europa Clipper missions and used for the majority of all system level tests including all testing with the flight vehicle.

TABLE OF CONTENTS

1. INTRODUCTION.....	1
2. GENESIS OF INGENIUM.....	1
3. INGENIUM CAPABILITIES.....	2
4. INGENIUM ARCHITECTURE.....	5
5. PROJECT ADOPTION/INTEGRATION.....	6
6. MISSION ENABLING: RIGOR AND EFFICIENCY	7
7. FUTURE WORK.....	7
8. CONCLUSION.....	8
ACKNOWLEDGEMENTS.....	8
REFERENCES.....	8
BIOGRAPHY	8

1. INTRODUCTION

Ingenium is a software application designed and developed at JPL to enable fully executable, electronic test procedures for system integration and test. Born out of a myriad of testing method and formats, it strives to standardize tests across projects and teams. In addition to standardization, it provides a no code method of writing fully executable (and automatable) test procedures while integrating with common data sources used in test procedure development. This is achieved via server application, composed of a suite of individual services, and client application which is able to directly interact with the existing Ground Data System (GDS) and Simulation and Support Equipment (SSE). Initially

released in 2019 it now handles the majority of system level test procedures in both simulation and hardware test venues for the Psyche and Europa Clipper projects. Its use has enabled a number of enhancements in both test efficiency and analytics while supporting project specific adaptations that tailor the capability to a given project. With extensive experience supporting system integration and test operations, Ingenium is now looking to support both subsystem level testing and on-console procedure for flight operations.

2. GENESIS OF INGENIUM

Prior to the development of Ingenium, tooling and methods for system test procedures were largely ad-hoc and varied between projects and individual teams on projects. This included both no-code, low-code and code-based solutions and was often driven by the individual team's experience and skill base. The existing solutions were not part of any institutional effort and relied on project personnel to develop and maintain them. This resulted in a patch work of approaches that often-required procedures to be reworked as they moved from one test venue to another and personnel to be re-trained as they switched between teams and projects.

The different approaches were primarily driven by two factors: software development experience and the level of formality of the test venue. While software based, scripted testing is common for many subsystems (e.g. flight software teams), that same skill set is not widely available in system level test teams and personnel. This caused issues with procedure development, procedure portability between individuals and teams, and procedure management and review. Development of these methods was largely ad-hoc and there was no institutional support for maintaining them across projects and teams. As a result, each project (and team) would tailor an approach based on the skills and experiences of their personnel.

The other driving factor, the formality of the test venue, is directly connected to the level of hardware and personnel safety concerns specific to that venue. Operating in System Integration and Test (SI&T) with flight hardware has both significant process and policy implications, as well as inherent conservatism. This is very different from operating in a simulated or partially simulated environment that does not utilize flight hardware. An example of the impact of policy differences is that the SI&T team always follows the

process for formally capturing tactical changes (e.g. redlines) to an executing procedure, while this is not always followed in other venues (in part because the venue is being used to develop the procedure). Automatic execution is another area where SI&T varies from other venues as its use is very limited given the concerns of working with, and possibly damaging, flight hardware. As a result, the vast majority of their test execution is performed with an operator in the loop on a step by step basis. The following is an overview of the different testing methods used for JPL in-house, system level test procedures: (over the past 10-15 years)

- Paper/Microsoft Word (No-Code)
 - Classical test procedures written as step by step instructions to be manually executed with the results manually recorded on paper during test execution.
- Python Script via Unix/Linux Console (Code)
 - Python based scripts, executed on the GDS workstation that interfaced directly with GDS and SSE. Allowed full test automation but required some experience with software development.
- Mars Science Laboratory Cross-Cutting Automated Script Process Framework – (MSL CASP) (Low-Code)
 - This was an early effort to standardize formats and reduce the level of code required to implement automated/scripted test procedures. It supported the generation of a python scripted test based on inputs provided in a Microsoft Excel file.
- Robot Framework (Low-Code)
 - An open source test automation framework was adapted support common test actions (based on the libraries developed in python scripts). It reduced the level of required software development experience and served to constrain procedure activities.
- Python Script via Jupyter Notebook (Code)
 - Python base scripts, executed via Jupyter Notebooks on the GDS workstation (using the same interfaces/libraries as python scripts). It provided richer output formats and step by step execution, and required some software development experience.
- “Baseline” Tests (Code to develop / No-Code to execute)

- These were a suite of system level regression tests, developed in either as python or Robot Framework scripts. Additional software tools were developed by individual projects to provide a level of monitoring, control, ease of use, and visibility not available in the native scripting methods. Often required extensive software development experience to build but little to none in order to execute.
- Baseline tests were primarily used in more formal test venues like SI&T which limited the use of automated test and had heightened hardware and personnel safety concerns. Because of their development effort they were typically limited to a single test suite per project.

These previous efforts both drove and informed the development of Ingenium. Many of them attempted to reduce the required software knowledge across the teams while keeping much of the inherent efficiency of automated testing. For more formal test venues, previous works strived to provide full control to test operators, ensure system visibility, and adhere to rigorous institutional processes designed with hardware and personnel safety in mind. Ingenium borrowed from all of these methods and created a solution that can be used across multiple JPL flight projects.

3. INGENIUM CAPABILITIES

Building upon past efforts, Ingenium endeavored to develop a system that combined the power and efficiency of scripted test procedures while keeping the associated learning curve as low as possible. It was also critical that the system would support the full spectrum of system test venues and, when required, would enforce the rigorous processes required for flight hardware testing. On top of these core capabilities, reporting and review functionalities were created that leverage a common data model of test data. As a multi-mission initiative, Ingenium does not intend to provide all functionalities that would meet every use case across all projects and teams. Rather, Ingenium provides core capabilities that are common to flight projects, and provide extension points that can be used by project teams to tailor Ingenium capabilities to specific needs of projects.

Executable test procedures that can directly interact with a test venue are critical to improving both the efficiency and reliability of testing. By automating the manual actions of test operators to input commands and then record and evaluate the resulting telemetry, the speed and reliability of test execution increases. This allows for more exhaustive telemetry predicts (which can detect off-nominal behavior), more frequent regression of test suites, and enhanced reporting and analytics of the results via a common data model. Automating much of the manual operations required to execute the test also frees the engineers to focus more on the health and performance of the vehicle under test. Ingenium needed to accomplish this automation without

incurring software development on the part of test engineers who implement such tests.

First, an extensive review of existing test procedures was done across a variety of formats and test venues. Individual actions in these procedures were categorized and prioritized by both their use and applicability across the spectrum of JPL missions. This resulted in a menu of procedure steps that encompassed over 80% of reviewed content while providing the ability to extend the capability via test operator actions or project specific code. The following is the list of built in Ingenium steps:

- Organization
 - Define a section that may contain steps and child sections
 - Define a text “paragraph”
 - Define a reference to another procedure
- Verification and Validation (V&V)
 - Define Verification Items (Requirements) to be verified by the procedure
 - Relate specific Verification Items to procedure steps to derive a pass/fail status
- Telemetry
 - Query for (or wait for) time series telemetry and evaluate it against predicts
 - Query for (or wait for) event telemetry (log messages) and evaluate it against predicts
 - Query for (or wait for) 1553 bus telemetry and evaluate it against predicts
 - Query for (or wait for) downlinked spacecraft produced files
 - Define reference times in a procedure to bound telemetry queries
- Commands
 - Send command(s) to the spacecraft
 - Send file(s) to the spacecraft
 - Send command(s) to the SSE
- Project Specific Functionality
 - Execute project specific
- Manual
 - Connect the test procedure with executing test data in the GDS
 - Accept operator input and evaluate it against predicts
 - Capture an operator action (with supporting inputs) and confirm its success
 - Wait a specified amount of time
 - Capture the current test venue metrology (temperature/humidity)
 - Accept operator input for electrical integration tests (voltage/resistance) and evaluate it against predicts

These steps would be assembled into test procedures using an interface that virtually eliminated the need for software development skills and included a variety of functions to speed procedure development. Users can drag and drop procedure steps into an interface that was designed to mimic a traditional paper procedure (Figure 1). Ingenium then assists the user in populating each step by looking up project data sources, such as command and telemetry dictionary and requirements. Procedure authors can copy and move steps, search and replace content, and perform procedure validation.

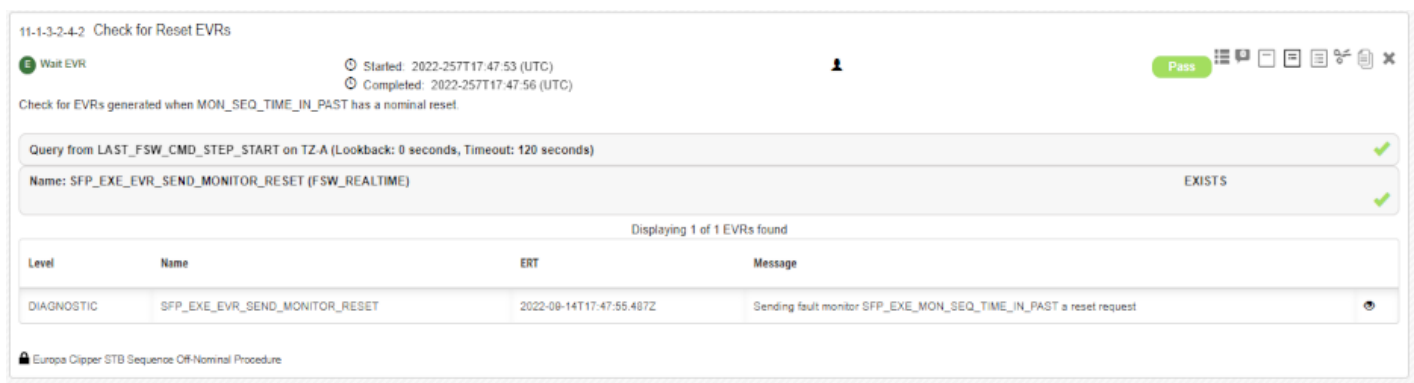


Figure 1 Sample Executed Step from an Ingenium Procedure

Test procedures rely on various project specific data including dictionaries of commands and telemetry, and the verification and validation objects that the procedures are developed against. Ingenium ingests this information and provides to the procedure author as they are writing the procedure, speeding accurate entry of this content. This

information can also be used to provide real time feedback to the user regarding the validity of their input. In addition, to better integrate with common data formats, each step is able to accept pasted content from spreadsheets and word processors.

11-1-4-6 Check Sequence Activation Count

Add tag...

Verify EHA

Check the initial sequence activation count (to compare against later)

Query from CURRENT_TIME (Lookback: 0 seconds, Timeout: 60 seconds)

<Channel SEQ

On TZ-A VALUE (DN) Record

ID>

Flight/SSE *

Flight

Data Path *

TZ-A

Channel ID *

Lookup

Channel Name *

SEQ

DN/EU Path *

DN

Verify On *

Value

Verify Condition

Record

+

Copy

SEQ_ACTIVATION_FAILURE

SEQ-0162

This channel contains a count of the number of times that a sequence has failed to be activated (regardless of the method).

SEQ_ACTIVATION_SUCCESS

SEQ-0161

This channel contains a count of the number of times that a sequence has been successfully activated (regardless of the method).

SEQ_COMPLETED_SEQUENCES

SEQ-0167

This channel contains a count of the number of sequences that have run to completion.

SEQ_DEACTIVATED_SEQUENCES

SEQ-0168

This channel contains a count of the number of sequence that have been deactivated (regardless of method).

SEQ_ENGINE_0_CMD_CNT

SEQ-0033

This channel contains the number of commands that have been dispatched in sequence engine 0.

SEQ_ENGINE_0_CMD_NUM

SEQ-0065

This channel contains the number of the last command dispatched from the sequence currently active in sequence engine 0.

11-1-4-7 Adjust Spacecraft Time

Add tag...

Cmd

Jump Spacecraft Time (SCLK) to ~30 seconds before EPOCH 0

Send Commands: TZ-A

FSW

CMD:

Verify: false

String: DEFAULT

Figure 2 Auto-complete of telemetry channels

Once a procedure has been drafted, Ingenium provides a number of features to support its review and revision. Integrated, step-by-step comments allow for concurrent, detailed review of a procedure. To respond to these comments and to aid in more systematic edits a search and replace function is provided, which is sensitive to the various fields and characteristics of Ingenium steps. A procedure validation capability is also provided to check the procedure content against project data (like requirements) and procedure references, speeding the update of older procedure content. Procedures and test executions can also be exported as PDFs and detailed ‘diff’ reports can be generated to illustrate the difference between procedure versions and different test executions.

A key requirement of Ingenium is to align with the formality of the test venue as this can vary substantially from simulator to flight hardware. This is supported via different modes of execution ranging from manual (step by step) to fully automatic step execution and the selective enforcement of procedure change documentation (Red/Blue lines). Ingenium allows steps to be executed individually or will

execute them automatically in procedure order with a variety of automatic halt conditions to the execution, including user defined break points. This allows for rapid execution for test development on simulators with much more constrained execution on hardware test venues. Similarly, if selected by the user, Ingenium can enforce rigorous procedure change documentation as required for official test results and formal test venues, while skipping that requirement during development in simulated environments. By allowing the test conductor to control the level of formality, Ingenium can support the full range of test venues.

The standardization and centralization of procedure and test execution information improved project and team interoperability as well as enabled a number of cross-cutting capabilities. The common interface and format have reduced the personnel training and made procedure content portable between teams and projects. This standardization, supported by Ingenium REST APIs and python SDKs, has enabled a suite of both standard and project developed tooling that leverage the data within Ingenium to analyze and validate the data while streamlining testing processes.

4

4. INGENIUM ARCHITECTURE

The Ingenium application has two primary components: a web application composed of a suite of microservices, and an agent application that resides on the GDS workstation of each test venue. The server application is composed of a rich user interface, implemented via a number of single page applications in the Vue.JS framework, and a number of

backend services and datastores. All backend services implement RESTful APIs per the OpenAPI (2.0) standard and are containerized using Docker. The agent application is a lightweight webservice that provides a secure RESTful interface to GDS and SSE functions on the GDS workstation. This agent is designed to directly interface with AMMOS Mission Data Processing and Control System (AMPCS) which is the standard GDS software suite for JPL missions. The services are illustrated in the following figure (Figure 3).

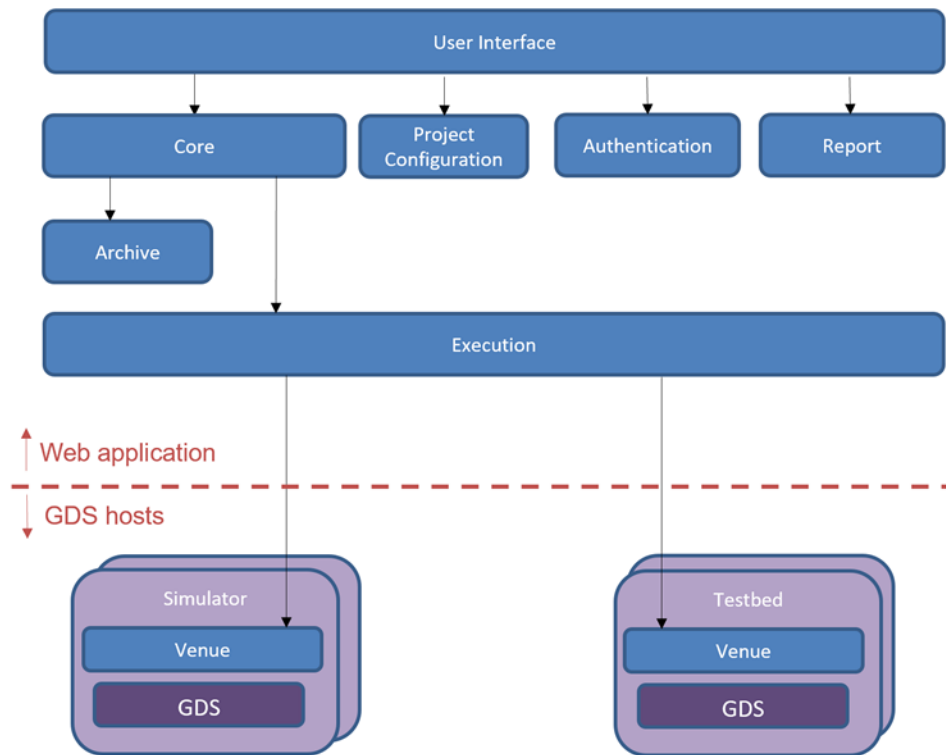


Figure 3 Ingenium Architecture

The services are:

- User interface: Provides a rich Web-based user experience for procedure authoring and execution that mimics paper procedure environment with enhanced drag and drop capability.
- Core: Primary backend service that manages procedures and executions
- Project Configuration: Caches project specific information, including requirements and command/telemetry dictionaries in an Elastic Search database.
- Authentication Service: Provides authentication via institutional services and authorization within the application
- Archive Service: Handles storage of procedure and execution data in an ArangoDB graph database.
- Execution Service: Executes each step in a procedure execution and handles the auto-execution logic.
- Report Service: Produces reports based on procedure and execution information including PDF, “Diff” reports, and filtered execution reports.
- Venue Service: Agent application that links the Ingenium server application and the GDS. Also executes project defined code for custom steps.

5. PROJECT ADOPTION/INTEGRATION

Ingenium was first installed in a production capacity supporting simulation and testbed in March of 2019 and installed in flight hardware venues in March of 2021. As of September 2022, the test campaign of one project (Psyche) is nearing completion, another project (Europa Clipper) is in process, and a third project (Mars Sample Retrieval Lander) has just started. These projects have generated over 15,000 test executions across simulator and non-flight hardware test venues (Figure 4), including approximately 1,700 executions performed against flight hardware (Figure 5).

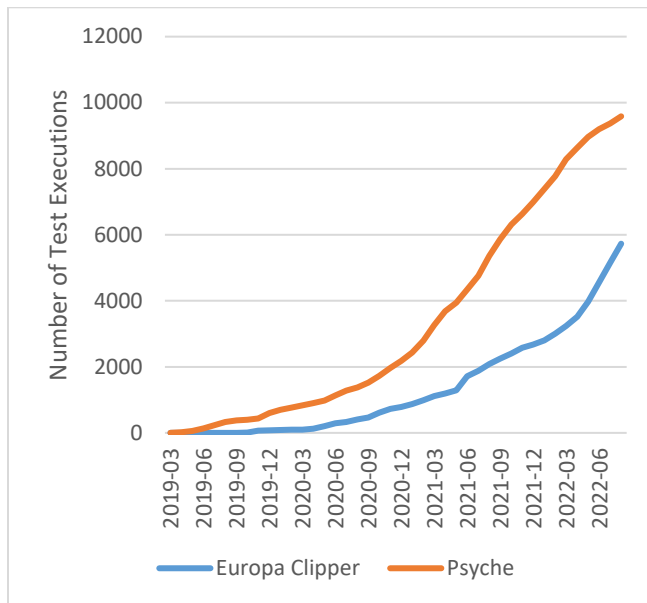


Figure 4 Ingenium Executions (Simulator and Testbed)

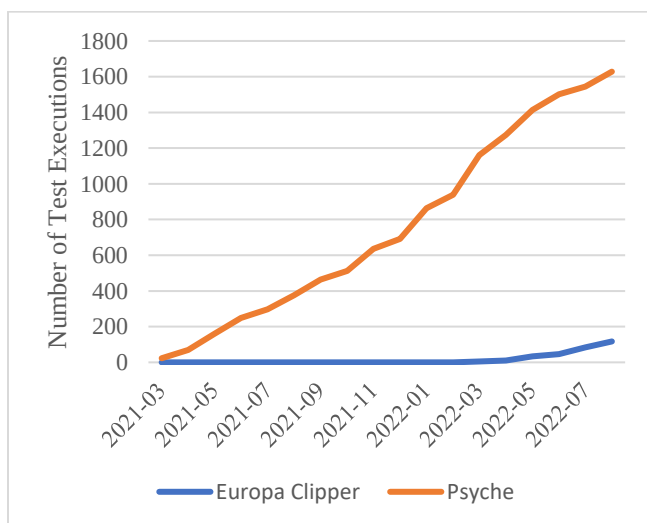


Figure 5 Ingenium Executions (Flight Hardware)

Figure 6 and Figure 7 show the growth of procedures and their revisions, both informal versions and formal versions

approved and released. The Psyche project (Figure 6) has completed much of their primary V&V campaign and their procedures and revisions have started to taper off while Europa Clipper (Figure 7) is still growing rapidly.

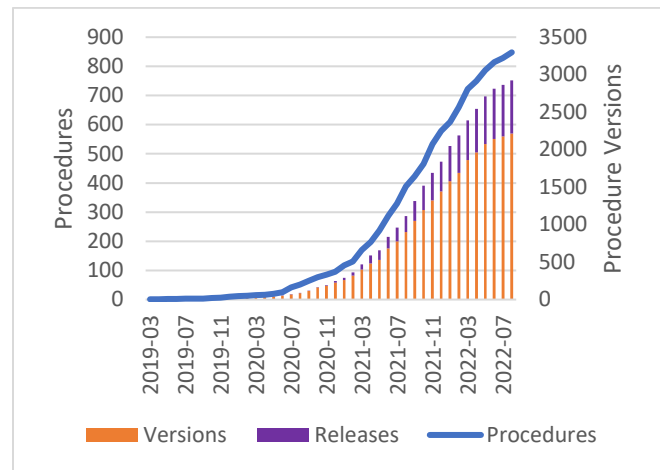


Figure 6 Psyche Procedures and Versions

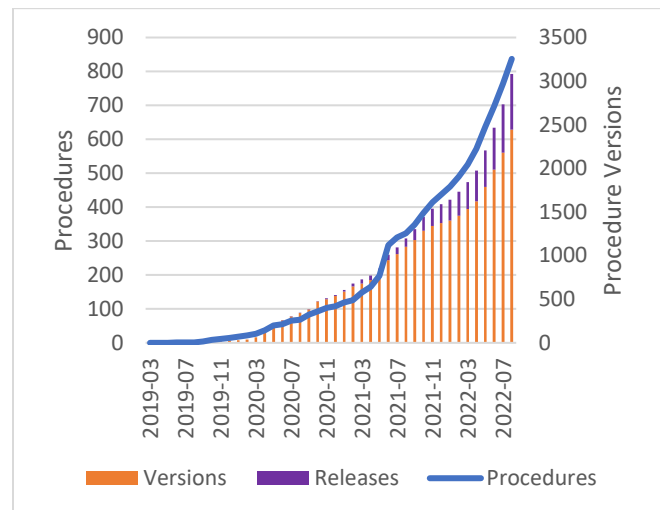


Figure 7 Europa Clipper Procedure and Versions

As part of the extensive procedure development and executions, both Psyche and Europa projects extended Ingenium capabilities to support project specific needs. Figure 8 shows the number of custom script steps developed so far by the two projects. Another area of custom development was pre/post processing and reporting capabilities that leveraged Ingenium API and SDK such as generation of integrated GDS session reports and automatic disposition of warning events.

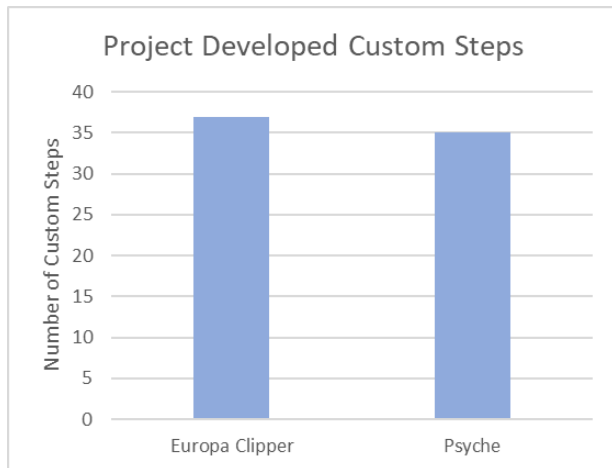


Figure 8 Project developed custom steps

6. MISSION ENABLING: RIGOR AND EFFICIENCY

The use of Ingenium has proven mission enabling in many areas, transforming many test actions and analyses from unique to standard and common place. The most significant benefit to flight projects has been that test engineers can execute test procedures in a more rigorous and efficient way. For example, Ingenium automates the verification of requirements via defined step predicts and a mapping of requirements to step results (Figure 9). This approach can significantly reduce the time required to review the data and record the results by automating much of the post-test analysis. It has the additional benefit of defining the success and failure criteria for requirements within the test procedure so these can be clearly understood and agreed upon prior to test execution. This is applicable to majority of test cases, while additional manual analysis may be required for more complex test cases with sophisticated verification criteria.



Figure 9 Requirements mapped to procedure steps

Another example is the removal manual inputs and verifications during procedure executions. Without using tools like Ingenium, test engineers will have to manually transcribe commands from paper to screen, check and record telemetries to verify pass/fail conditions of steps. These manual actions are tedious and error prone. Since these actions are automated by Ingenium, this translates into substantial efficiency and improved rigor as the ATLO team no longer needs to wait for an operator to complete the manual actions. An excellent case study of this efficiency is querying for time series telemetry (temperatures, voltages, etc.) and comparing the results against a predicts. It is one of the most common actions in a test procedure and during the Psyche ATLO campaign it was performed 372,762 times as of September 2022. Assuming that these measurements would take test engineers several seconds to manually perform, this translates to over a month of test shift time. By automating error prone manual checks, Ingenium also improves the rigor of integration and tests campaign.

The standard electronic format and available interfaces to the data has created opportunities for projects to extend Ingenium

capabilities. The Europa Clipper project leveraged the Ingenium SDK to create an external validation of test procedures for flight hardware test venues. This specifically focused on querying procedures for all commands used, and then comparing them to an externally provided list of commands the have been previously tested and approved for use on flight hardware. While tools like this was possible in the past they were both more difficult to implement and much less reliable as they relied on pattern matching and had to be implemented for each format the project supported.

7. FUTURE WORK

As the adoption of Ingenium for system level test campaigns have increased, so has interest in applying the same approach to procedures in other environments. This includes supporting on-console procedures during mission operations (Europa Clipper) and support for select subsystem testing (Mars Sample Return). While each environment brings different challenges that may require further development, each team saw the value in the demonstrated capability for system tests and selected it for use in their environments.

8. CONCLUSION

The Ingenium application has brought a common framework to system testing at JPL bringing new levels of efficiency and rigor to test campaigns across projects and teams. Leveraging past automation efforts and interfaces it is able to directly execute test actions in all system level test venues. It has been able to provide a suite of multi-mission capabilities that address the majority of projects needs while providing interfaces for project specific extensions. Since its introduction it has seen rapid adoption on the major flight projects in development at JPL and has delivered substantial benefits at the intuitional, project, and team level. Building on its success in system testing it is now being applied to environments in both mission operations and subsystem level testing.

ACKNOWLEDGEMENTS

The research was carried out at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration.

Ingenium would like to acknowledge the following individuals and teams for their support:

- Rich Kinslow, Shaun Standley, Alex Jimenez, Todd Bayer, Jim Chase
- The Ingenium development team
- The Europa Clipper and Psyche Project integration and test teams.

REFERENCES

- [1] <https://robotframework.org>, cited in September 2022
- [2] <https://jupyter.org>, cited in September 2022.
- [3] <https://www.openapis.org>, cited in September 2022.
- [4] <https://docs.docker.com>, cited in September 2022.
- [5] <https://vuejs.org>, cited in September 2022.

BIOGRAPHY



Christopher Swan received a B.S. in Aerospace Engineering from Illinois Institute of Technology in 2002 and a M.S. in Astronautical Engineering from USC in 2007. He is currently serving as both the Ingenium Task Lead and the Europa Clipper Behavior Lead. Prior to his current role, he worked as the Behavior Lead on the SMAP project, later moving into engineering leadership as the Science Phase Lead during flight operations. Additionally, he was a surface and payload system engineer on the Phoenix Mars mission,

supporting both System I&T and surface operations.



Hongman Kim is a systems engineer at NASA's Jet Propulsion Laboratory. Since he joined JPL in 2014, Hongman worked on a number of software development projects including concept design study environment and ontology-based engineering data integration. Currently, he works as software architect of Ingenium that provides a Web-based test procedure authoring and execution environment for system level testing and integration of JPL flight projects. Before joining JPL, he was a project manager at Phoenix Integration, Inc., where he led development of model-based systems engineering (MBSE) technology. He also worked on a number of government funded projects including points cloud visualization, distributed computing, and design optimization. He holds a PhD degree in aerospace engineering from Virginia Tech, and MS and BS degrees in aerospace engineering from Seoul National University.